Practical Work No. 15
DS Queues

1. Définition

A queue is a set consisting of a variable, possibly null, number of data elements, on which the following operations can be performed:

- **create_queue**: creates an empty queue (creation).
- **is_empty**: checks whether a queue is empty or not (consultation).
- **enqueue (enfiler en Fr)**: adds a data element of type T to the queue (modification).
- **dequeue (defiler en Fr)**: remove the oldest element from the queue (modification).
- **first (premier)** : retrieves the oldest element in the queue (consultation).

Illegal operations: Some operations defined on the queue data structure require preconditions:

- **dequeue** requires that the queue is not empty.
- **first** requires that the queue is not empty.

2. Properties

In this paragraph, we will list the properties that characterize the semantics of the operations applicable to the queue data structure.

- **F1**: **create_queue** creates an empty queue.
- **F2**: If an element is added (**enqueue**) to the resulting queue, it is not empty.
- **F3**: An element added (**enqueue**) to the queue immediately becomes the first if the queue was empty; otherwise, the first element remains unchanged.
- **F4**: A successive enqueue and dequeue operation on an empty queue leaves it empty.
- **F5**: A successive enqueue and dequeue operation on a non-empty queue can be performed in any order.

Illustration:
Non-empty queue: 5 6 1

- **enqueue(8)** → 5 6 1 8
- **dequeue()** → 6 1 8

The queue structure follows the **FIFO** rule: **First In, First Out**.

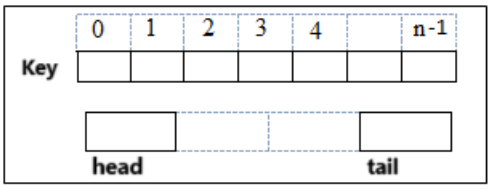
3. Physical Representation of a Data Structure

To materialize (concretize, implement) a data structure, two types of representation are distinguished:

- **Linked Representation**: The elements of a data structure are placed at arbitrary locations (in main memory) but are linked together. ➔ pointers.
- **Contiguous Representation**: The elements of a data structure are stored in a contiguous space. ➔ array.

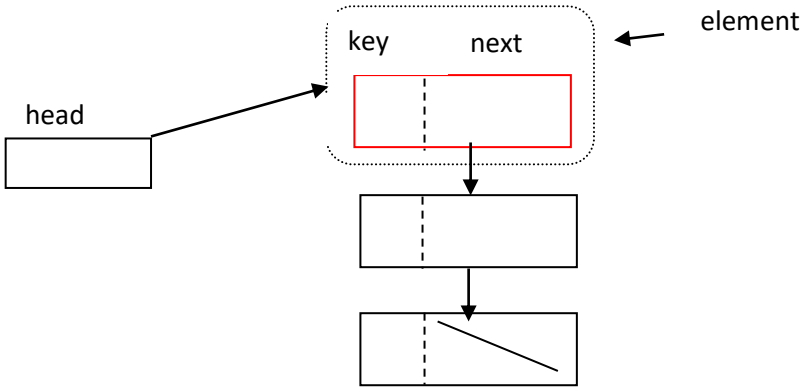
3.1 Contiguous Representation

- TDA Queue Implemented with a Contiguous Representation
- It consists of an array with two entry points: two indices, **head** and **tail**.




```
const  int N =100 ;
struct Queue{
        int key [N] ;
        int head ;
        int tail ;
};
```

3.2 Linked Representation

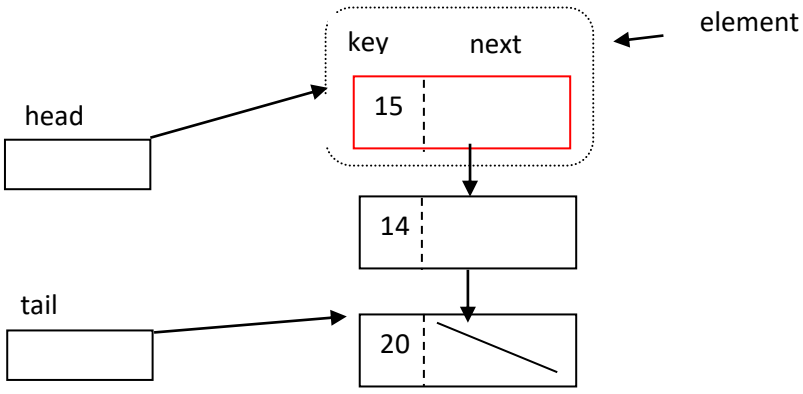


- **first**: costs one indirection from the head pointer.
- **dequeue**: costs one indirection from the head pointer.
- **enqueue**: costs traversing the entire queue from the head pointer.

This requires multiple indirections. Thus, the proposed solution should not be retained.

Solution: A physical representation with two entry points, **head** and **tail**, is needed.

- The **head pointer** enables efficient implementation of the **first** and **dequeue** operations.
- The **tail pointer** enables efficient implementation of the **enqueue** operation.



Translation in C :

```
struct Element {
  int key;
  struct Element* next;
};

struct Queue {
  struct Element* head;
  struct Element* tail;
};
```

Note:

The queue data structure is structured with two entry points. In contrast, the stack data structure is a structure with a single entry point.

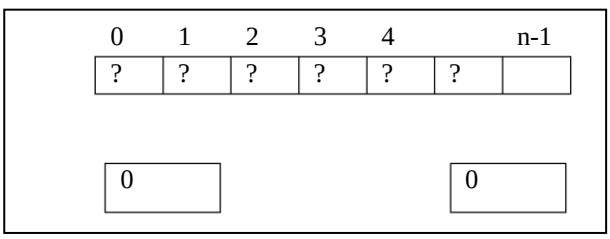
4. Materialization of the Queue Data Structure as an Abstract Data Type with a Contiguous Representation

/* Linked representation */ **#define n 100** **struct Queue {** **int key[n];** **unsigned head;** **unsigned tail;** **};**	**void create_queue(struct Queue*);** **/* struct Queue* create_queue(void); */** **unsigned is_empty(struct Queue);** **/* unsigned is_empty(struct Queue*); */** **int first (struct Queue);** **/* int first (struct Queue*); */** **void enqueue(int, struct Queue*);** **void dequeue(struct Queue*);**

The problem posed by the contiguous representation of the SD file:

Question: Starting from this empty queue, execute the following sequence:

key



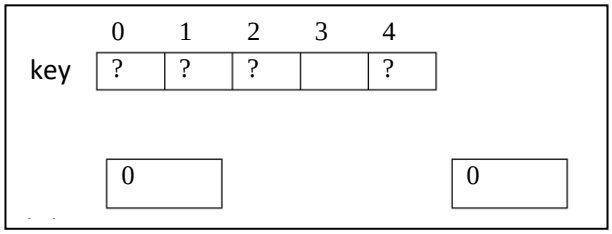
enqueue the elements: 100, 20, 30, 40, 13;

dequeue

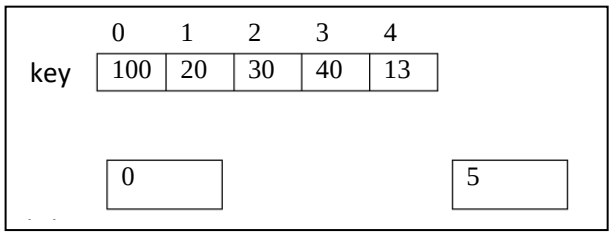
enqueue 120

Résultat :

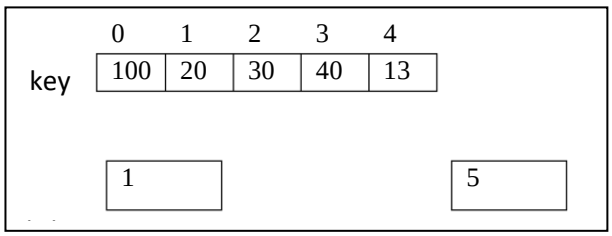
a) Initial situation:



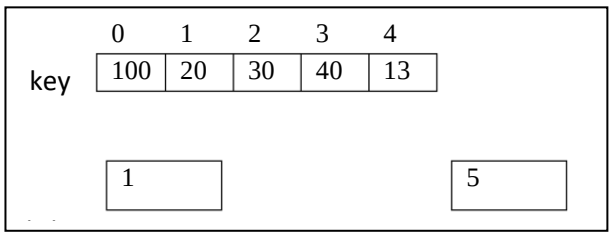
b) Situation after the 5 enqueues:



c) Situation after the dequeue:



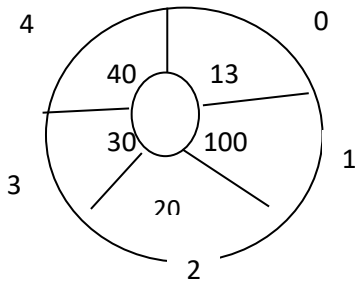
d) Final situation: The element at position 0 is available



You cannot enqueue 120 after the queue (indeed, the element at position queue (5) does not belong to the key array. Yet, the queue is not full!! This is because the array is perceived in a linear way, it is traversed from left to right. After n enqueues, you can no longer add new elements, even if you dequeue. Here, n is the size of the key array.

Solution: a circular array, that is, head and queue modulo n, with n being the size of the array. This is a logical, not physical, perspective. We will apply the sequence of actions a, b, c, previously seen on a linear array, on an array perceived as circular.

Initial situation: empty queue.



head 0

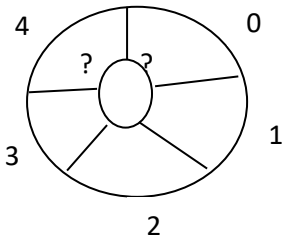
tail 0

Convention: We enqueue after the queue.
We dequeue: after the head.

Enqueue 100	queue=1
Enqueue 20	queue=2
Enqueue 30	queue=3
Enqueue 40	queue=4
Enqueue 13	(queue +1 > 4) ➔ queue=0
Dequeue ➔ ~~100~~	head=1

Enqueue 120 queue+1=0+1=1 ➔ this position is available. The element in this position has already been processed in (b).

Constatation :



head 0

tail 0

head = tail → empty queue or full queue

→ This leads to a problem.

Idea:

Instead of reserving an array (here key) of n elements, we plan an array of size n+1, while respecting the following property:

- We use at most n elements: when the queue contains n elements, the queue is logically full but not physically full.
- The condition head = tail characterizes an empty queue.

/***Implementation queue.c ***/**

```
#include <assert.h>
#include "queue.h"

void create_queue(struct queue *q) {
   q->head = 0;
   q->tail = 0;
}

/*RQ: Any index between 0 and n-1 will work.  */

unsigned is_empty (struct queue q) {
   return (q.head == q.tail);
}

int first(struct queue q) {
   unsigned i;
   assert(!is_empty(q));
   i = q.head + 1;

   if (i > n - 1)
      i = 0;

   return (q.key[i]);
}

void enqueue (int info, struct queue *q) {
   q->tail++;

   if (q->tail > n - 1)
      q->tail = 0;

   assert(q->head != q->tail);
   q->key[q->tail] = info;
}

void dequeue(struct queue *q) {
   assert(!is_empty(*q));
   q->head++;

   if (q->head > n - 1)
      q->head = 0;
}
```

5. Application Exercises:

Exercise 1:

Implement in C the data structure Queue as an Abstract Data Type (ADT) using a linked list representation.

Exercise 2:

Implement the method `void dequeueUntil (struct Queue *Q, int x)` that dequeues elements from the queue until reaching element `x` or until the queue is empty. **Note:** The element `x` is not dequeued.

Exercise 3:

Based on the Stack data structure already covered in class (see `stack.h` interface), we aim to implement the Queue data structure using two stacks. To achieve this, complete the source file `queue.c` as well as the test file `test.c`.

/******* Interface part: stack.h *******/

```
/* physical representation */
struct element {
  int key ;
  struct element* next ;
} ;
/* operations or exported services */
struct element* create_stack (void) ;
unsigned is_empty (struct element*) ;
int last (struct element*) ;
void push (int, struct element**) ;
void pop (struct element**) ;
```

/******* Interface part: queue.h *******/

```
#include "stack.h"

struct Queue {

   struct element * P_In;

   struct element * P_Out;

};

void create_queue(struct Queue*);

unsigned is_empty(struct Queue);

int first (struct Queue);

void enqueue(int, struct Queue*);

void dequeue(struct Queue*);
```